

UNIVERSIDAD NACIONAL DE LA PATAGONIA AUSTRAL
LICENCIATURA EN SISTEMAS • 4º AÑO
GESTIÓN DE CALIDAD

T.P. Nº

7.1

TRABAJO PRÁCTICO

Evaluación de Calidad Interna del Producto Software

AUTOR

Gargaglione, Giancarlo

FECHA DE
ENTREGA

26/05/2026

Resolución

1.

Analice la documentación de las herramientas Mess Detector y Code Sniffer. Describa de forma resumida en qué consisten y qué reglas evalúan. Elija las 5 reglas que considera más útiles y/o relevantes para el desarrollador.

PHP MESS DETECTOR (PHPMD)

PHP Mess Detector es una herramienta de análisis estático que examina el código fuente PHP en busca de problemas potenciales relacionados con la complejidad, el diseño, las buenas prácticas y el código no utilizado. Detecta posibles bugs, código subóptimo, expresiones demasiado complejas y parámetros, variables, métodos y clases sin uso. Opera aplicando conjuntos de reglas predefinidos o personalizados sobre los archivos PHP del proyecto.

PHPMD organiza sus reglas en seis categorías:

- **Codesize:** detecta código demasiado largo o complejo — métodos extensos, clases con demasiadas responsabilidades, alta complejidad ciclomática.
- **Design:** identifica problemas de diseño como dependencias excesivas o uso inapropiado de variables globales.
- **Naming:** verifica convenciones de nomenclatura — nombres de variables demasiado cortos o largos, nombres de métodos poco descriptivos.
- **Unusedcode:** detecta variables, parámetros, métodos y propiedades declarados pero nunca utilizados.
- **Cleancode:** promueve prácticas de código limpio, como evitar argumentos booleanos en funciones.
- **Controversial:** reglas de aplicación discutida, como el uso de superglobales.

Las 5 reglas más relevantes para el proyecto:

Regla	Categoría	Descripción
CyclomaticComplexity	Codesize	Detecta métodos con complejidad ciclomática alta, demasiados caminos de ejecución posibles. Generalmente 1-4 es baja complejidad, 5-7 moderada, 8-10 alta y 11+ muy alta. Relevante para los módulos de validación y ABM.

Regla	Categoría	Descripción
UnusedLocalVariable	Unusedcode	Detecta variables declaradas pero no utilizadas dentro de un método. Frecuente en código PHP sin tipado estricto.
UnusedFormalParameter	Unusedcode	Detecta parámetros de funciones que se declaran pero nunca se usan dentro del cuerpo. Indica código innecesario o funciones mal diseñadas.
ShortVariable	Naming	Alerta sobre variables con nombres demasiado cortos (menos de 3 caracteres) que dificultan la lectura. Relevante para el estándar de programación EC-STD-PHP.
BooleanArgumentFlag	Cleancode	No debe haber espacio entre el nombre de una función y el paréntesis de apertura. Garantiza uniformidad en todas las llamadas a funciones del proyecto.

PHP CODE SNIFFER (PHPCS)

PHP CodeSniffer es una herramienta que analiza el código PHP y detecta violaciones de los estándares de codificación definidos. A diferencia de PHPMD, que analiza la calidad del diseño, PHPCS se enfoca en el estilo y formato del código — indentación, espaciado, nomenclatura y estructura. Puede integrarse en el flujo de trabajo de desarrollo para verificar automáticamente el cumplimiento del código con los estándares. EdgewallEdgewall

PHPCS soporta múltiples estándares predefinidos, siendo PSR-1 y PSR-12 los más relevantes para este proyecto. PSR-12 es el estándar de estilo de codificación actualmente recomendado, habiendo reemplazado al anterior PSR-2. Todos los principales frameworks PHP han adoptado estos estándares como base.

Las 5 reglas más relevantes para el proyecto:

Regla	Estándar	Descripción
Indentation	Codesize	El código debe usar 4 espacios de indentación por nivel, nunca tabulaciones. Garantiza consistencia visual entre distintos editores.
LineLength	Unusedcode	Se recomienda mantener las líneas en aproximadamente 80 caracteres para mejorar la legibilidad. Relevante para el código PHP de los formularios y la lógica del ABM.
DisallowMultipleStatements	Unusedcode	No se permiten múltiples sentencias en una sola línea. Mejora la trazabilidad del código durante revisiones y depuración.

Regla	Estándar	Descripción
ControlStructureSpacing	Naming	Las estructuras de control (if, for, while) deben usar llaves siempre, incluso para bloques de una sola línea. Previene errores sutiles por omisión de llaves.
FunctionCallSpacing	Cleancode	No debe haber espacio entre el nombre de una función y el paréntesis de apertura. Garantiza uniformidad en todas las llamadas a funciones del proyecto.

2.

Analice el código de la aplicación desarrollada en el trabajo 05.2 con base a los estándares básicos de codificación de los lenguajes que ha utilizado (describa si lo hizo manual o automáticamente). Elabore un cuadro de acuerdo al siguiente ejemplo:

PSR	Recomendaciones	Evaluación	Sugerencias de mejora
1	2.1. Etiquetas PHP	Cumple	
1	2.2. Codificación	Cumple	
1	2.3. Declaraciones X Side effects	Clases de modelo incluyen salidas HTML	Refactoring. Mover código de salida HTML hacia una vista y utilizar getters en la vista.

El análisis se hizo manualmente, revisando el código línea por línea contra las recomendaciones PSR-1 y PSR-12 y el estándar EC-STD-PHP del proyecto.

PSR	Recomendaciones	Evaluación	Sugerencias de mejora
1	2.1. Etiquetas PHP — usar <code><?php</code> o <code><?=></code> únicamente	Cumple	-
1	2.2. Codificación — UTF-8 sin BOM	Cumple	
1	2.3. Side effects — un archivo no debería declarar símbolos Y producir salida a la vez	No cumple	El archivo ejecuta lógica de conexión y consulta, y además genera HTML. Separar en un archivo de configuración (config.php) y un archivo de vista (catalogo.php) que solo renderiza.

PSR	Recomendaciones	Evaluación	Sugerencias de mejora
12	Indentación: 4 espacios, sin tabulaciones	Cumple	-
12	Longitud de línea: máximo 80 caracteres recomendado	Cumple parcialmente	La línea del SELECT supera los 80 caracteres. Se dividió la query en variable multilínea con indentación consistente.
12	Estructuras de control: siempre con llaves	Cumple	Se usan sintaxis alternativas (if:/endif:) correctamente en contexto HTML.
12	Espaciado: espacio entre palabra clave y paréntesis	Cumple	-
12	Una declaración por línea	Cumple	-
EC-STD-PHP	Cabecera de archivo con PHPDoc	Cumple	-
EC-STD-PHP	Variables en camelCase	Cumple: \$pdo \$stmt \$muebles \$mueble	-
EC-STD-PHP	Comentarios explican el por qué, no el qué	Cumple parcialmente	El comentario “// Configuración de conexión” describe el qué. Podría eliminarse dado que el código es autoexplicativo.
EC-STD-PHP	Funciones documentadas con PHPDoc	No aplica	No hay funciones declaradas en este archivo, la lógica es procedimental.

3.

Elabore una tabla y relacione las recomendaciones PSR-1, PSR-12 y PSR-4 con los atributos de calidad descritos en ISO 25010.

PSR	Recomendación	Atributo ISO 25010	Justificación
1	Uso exclusivo de etiquetas <?php y <?= <?=>	Compatibilidad (Portability)	Evita etiquetas cortas (<?) que pueden estar deshabilitadas en algunos servidores, garantizando que el código funcione en distintos entornos.

PSR	Recomendación	Atributo ISO 25010	Justificación
1	Codificación UTF-8 sin BOM	Compatibilidad (Portability)	Asegura que el código se interprete correctamente en cualquier servidor y navegador, independientemente del sistema operativo.
1	Separación de side effects y declaraciones	Mantenibilidad (Modularity)	Un archivo con una sola responsabilidad es más fácil de modificar, reutilizar y probar sin efectos inesperados en otras partes del sistema.
1	Nombres de clases en PascalCase	Mantenibilidad (Modularity)	La consistencia en la nomenclatura reduce el esfuerzo cognitivo al leer código escrito por distintos desarrolladores.
1	Constantes en mayúsculas con guión bajo	Mantenibilidad (Readability)	Distingue visualmente las constantes de las variables, reduciendo errores de lectura e interpretación.
12	Indentación de 4 espacios sin tabulaciones	Mantenibilidad (Readability)	Garantiza que el código se visualice de forma consistente en cualquier editor, independientemente de la configuración de tabulaciones.
12	Longitud de línea máxima de 80 caracteres	Mantenibilidad (Readability)	Permite visualizar el código sin scroll horizontal, facilitando la revisión y las comparaciones de versiones en herramientas como GitHub.
12	Llaves obligatorias en estructuras de control	Confiabilidad (Fault Tolerance)	Previene bugs sutiles al agregar líneas a un bloque que originalmente tenía una sola sentencia sin llaves.
12	Una sola declaración por línea	Mantenibilidad (Analyzability)	Facilita la lectura del historial de cambios en Git: cada línea modificada corresponde a una sola acción, simplificando el diff.
12	Espaciado consistente en operadores y comas	Mantenibilidad (Readability)	Reduce el esfuerzo de lectura al separar visualmente los operandos y argumentos, especialmente en expresiones complejas.
4	Autoloading basado en namespaces y estructura de directorios	Mantenibilidad (Modularity)	Cada clase en su propio archivo con un nombre predecible permite cargar únicamente lo necesario, reduciendo el acoplamiento entre componentes.
4	Correspondencia entre namespace y ruta de archivo	Mantenibilidad (Analyzability)	Permite localizar cualquier clase del proyecto sin necesidad de buscar: la ruta del archivo se deduce directamente del namespace.
4	Un namespace por archivo	Mantenibilidad (Modularity)	Delimita claramente las responsabilidades de cada archivo, facilitando el reemplazo o la

PSR	Recomendación	Atributo ISO 25010	Justificación
			modificación de componentes individuales sin afectar al resto.

4.

Actualice el plan de SQA. Incorpore recomendaciones relacionadas al uso de estándares de documentación, estándares de codificación y herramientas para su evaluación.

El Plan de SQA del proyecto (EC-SQA-PLN) fue actualizado para incorporar las recomendaciones derivadas del análisis de calidad interna realizado en el presente trabajo práctico. A continuación se detallan los cambios introducidos.

Estándares de codificación:

Se incorporaron formalmente los estándares PSR-1 y PSR-12 del PHP Framework Interop Group (PHP-FIG) como referencias de codificación obligatorias para el proyecto, complementando el estándar de programación PHP definido en EC-STD-PHP. Estos estándares ya estaban mencionados en la sección 5.1 del Plan de SQA dentro de la tabla de estándares adoptados, pero se explicita aquí su alcance concreto:

- PSR-1 establece las convenciones básicas de codificación: etiquetas PHP, codificación de archivos, nomenclatura de clases y constantes, y separación de side effects.
- PSR-12 extiende PSR-1 con reglas de estilo detalladas: indentación, longitud de línea, espaciado, uso de llaves en estructuras de control y declaración de tipos.

Ambos estándares son verificables automáticamente mediante PHP Code Sniffer, incorporado como herramienta de soporte en la sección 8 del Plan de SQA.

Estándares de documentación confirmados:

El estándar de documentación inline del código se confirma en EC-STD-PHP: todo archivo PHP debe incluir una cabecera con PHPDoc, y toda función declarada debe estar documentada con su descripción, parámetros y valor de retorno. Esta práctica se alinea con el atributo de Mantenibilidad (Analyzability) de ISO/IEC 25010, al garantizar que el código sea comprensible por un desarrollador independiente del equipo original.

Herramientas de evaluación incorporadas:

Se incorporan dos herramientas de análisis estático a la sección 8 del Plan de SQA, ya listadas en la tabla de herramientas:

- PHP Code Sniffer (PHPCS): verifica automáticamente el cumplimiento de PSR-1 y PSR-12 sobre el código fuente PHP. Se ejecuta sobre la carpeta 05-implementacion/php/ antes de cada merge a main, como parte del proceso de auditoría funcional (FCA) definido en la sección 6.3 del Plan de SQA.
- PHP Mess Detector (PHPMD): detecta problemas de diseño, complejidad y código no utilizado. Se ejecuta con los rulesets codesize, unusedcode, naming y cleancode. Los hallazgos se registran como Issues en GitHub con etiqueta audit.

El análisis del archivo **index.php** realizado en la actividad 2 reveló dos incumplimientos parciales:

El primero corresponde a PSR-1, sección 2.3 (side effects): el archivo combina lógica de conexión a la base de datos con generación de HTML en un mismo archivo. La corrección propuesta es separar la configuración de la conexión en un archivo config.php independiente, reduciendo el acoplamiento y mejorando la modularidad del sistema.

El segundo corresponde a la longitud de línea: la consulta SQL supera los 80 caracteres recomendados por PSR-12. La corrección propuesta es dividir la query en una variable multilínea con indentación consistente.

Ambas correcciones fueron registradas como Issues en el repositorio GitHub del proyecto con etiqueta scr para su seguimiento formal.

Backlog Priority board Team items Roadmap My items + New view

Filter by keyword or by field

Pendiente 6 Estimate: 0

No iniciado

- abm-muebles-gz #8
Revisión Técnica Formal
- abm-muebles-gz #10
Login (RF-02 y 03)
- abm-muebles-gz #11
ABM de Muebles (RF-04, 05 y 06)
- abm-muebles-gz #12
Análisis PHPCS/PHMD
- abm-muebles-gz #13
Gestión de Categorías (RF-09)
- abm-muebles-gz #16
EC-STD-PHP - Estándar de programación PHP

En curso 0 / 3 Estimate: 0

Se está trabajando

En revisión 0 / 5 Estimate: 0

Esperando aprobación del otro integrante

+ Add item

1	✓	Estándar de documentación #1
2	✓	Estructura de carpetas #2
3	⊙	EC-STD-PHP - Estándar de progr... #16
4	✓	Actualizar Plan de SQA #6
5	✓	Requerimientos #3
6	✓	Revisión de requerimientos #7
7	✓	Modelo de datos #4
8	✓	Ver Catálogo (RF-01) #9
9	✓	Implementar BD #5
10	⊙	Revisión Técnica Formal #8
11	⊙	Login (RF-02 y 03) #10
12	⊙	ABM de Muebles (RF-04, 05 y 06) #11
13	⊙	Análisis PHPCS/PHMD #12
14	⊙	Gestión de Categorías (RF-09) #13
15	✓	EC-RSG-ANX - Análisis de riesgos #15
16	✓	EC-RSG - Plan de Riesgos #14

✓ Estándar de documentación #1

✓ Estructura de carpetas #2

⊙ EC-STD-PHP - Estándar de programación PHP #16

✓ Actualizar Plan de SQA #6

✓ Requerimientos #3

✓ Revisión de requerimientos #7

✓ Modelo de datos #4

✓ Ver Catálogo (RF-01) #9

✓ Implementar BD #5